



Afi Global Education

MÁSTER EN FINANZAS CUANTITATIVAS 2025-2026

EJERCICIO DE SIMULACIÓN: DESARROLLO Y PUESTA EN
PRÁCTICA DE UNA ESTRATEGIA DE GESTIÓN
CUANTITATIVA

Memoria explicativa

Pedro Estévez García
Juan Cristóbal Olmedo Moreno
Javier Puerta Gómez
Paula Simón Ávila
Carlos Zaragoza Navarro

28 de mayo de 2026

Índice

1. Introducción	2
2. Metodología aplicada	4
2.1. Modelos ML	4
2.1.1. Tipo de aprendizaje y enfoque del problema	4
2.1.2. Modelo seleccionado: XGBoost (Extreme Gradient Boosting)	4
2.2. Evaluación. Métricas de rendimiento y riesgo	5
2.3. Desagregación de resultados	5
3. Sistema e infraestructura del modelo	8
3.1. Estructura del bloque de código principal	8
3.2. Estructura de código de análisis de resultados	9
4. Backtests e implementación de la estrategia	13
5. Resultados y análisis de la estrategia	14
6. Conclusiones	16
7. Anexo	17
7.1. Backtest	17

1. Introducción

La gestión cuantitativa de activos financieros es un enfoque de inversión que utiliza modelos matemáticos y algoritmos computacionales para gestionar una cartera de activos financieros. Para ello, se deben aplicar criterios estrictamente objetivos basados en el análisis de datos y modelización matemática avanzada [2].

Así, un modelo de gestión cuantitativa puede incluir criterios de selección de activos, modelos de riesgo, optimización de carteras, validaciones por backtesting y sistemas de ejecución sistemática según las reglas definidas. Dado que se trata de una definición amplia, existe una gran variedad de estrategias que se pueden calificar como de gestión cuantitativa. La variedad está, entre otras características, en la elección del subyacente, el tipo de activo financiero, los datos utilizados para tratar de predecir los movimientos del mercado, la metodología matemática para modelizar estos cambios, etc.

Nuestro grupo ha apostado por una estrategia de stockpicking sobre el índice EuroStoxx50, es decir, trata de predecir qué acciones del EuroStoxx50 van a tener un mejor desempeño e invierte en ellas. Desde un punto de vista matemático, la metodología más directa que se puede plantear para tratar de predecir rendimientos es realizar algún tipo de regresión. Sin embargo, consideramos que es más adecuado afrontar el problema como una clasificación binaria: en lugar de preguntarnos cuál será el retorno de una acción en el futuro próximo, nos preguntamos si dicha acción quedará en el top N (o superará la mediana) entre sus peers en términos de rentabilidad.

Por tanto, lo anterior se transforma en un problema supervisado de clasificación binaria con múltiples variables explicativas. En este punto, la primera idea es aplicar un modelo de regresión logística: utilizamos datos históricos para obtener los coeficientes del modelo. Esto nos permitirá no solo clasificar cada activo según la variable objetivo definida anteriormente, sino que también permite el cálculo de unas “probabilidades”: un score que permite ordenar los activos según una estimación de la probabilidad de pertenecer a la clase positiva.

Aunque un modelo de regresión logística es una herramienta con la que nos sentimos cómodos debido a la educación de los integrantes del grupo, nos dimos cuenta de que presenta algunos inconvenientes para este problema en particular. La regresión logística sería una opción restrictiva porque asume una relación lineal entre las variables y los logaritmos de las probabilidades de pertenecer a cada grupo de la clasificación binaria. Esto es relevante en este contexto porque es razonable pensar que la relación entre las variables de mercado generalmente utilizadas y las rentabilidades futuras no sea lineal ni estable. Además, la regresión logística no detecta las relaciones entre las variables de manera automática.

En consecuencia de lo anterior, decidimos implementar un modelo de clasificación no lineal basado en árboles de decisión porque resolvían los principales problemas comentados: contemplan relaciones no lineales entre las variables del modelo y la variable objetivo. Además, los modelos basados en árboles pueden manejar de forma más flexible conjuntos de variables redundantes o relacionadas entre sí, sin exigir una relación lineal e independiente de cada variable con la variable objetivo.

En este contexto, no se pretende que el modelo estime una predicción exacta, sino una señal relativa para ordenar los activos y seleccionar candidatos para la cartera. Una vez decidido el mecanismo para la selección de activos, nos encontramos con la decisión de asignar pesos a cada uno de los activos seleccionados por el modelo. Originalmente, optamos por una distribución equiponderada por simplicidad, mientras experimentábamos con distintos mecanismos de elección de pesos.

Los principales métodos de elección de pesos considerados fueron: según los scores del modelo, según ciertos parámetros macroeconómicos, según las correlaciones entre los activos para minimizar la varianza o mediante simulaciones de Montecarlo para optimizar un parámetro como el sharpe. Descartamos la primera opción porque implicaría darle el doble de peso a las decisiones del modelo, y no estábamos tan cómodos con el ratio de acierto del mismo durante los backtests. La segunda opción también la descartamos porque los datos macroeconómicos que a los que se pueden acceder de manera gratuita son limitados, y los backtests mostraban resultados inferiores a las otras estrategias.

Finalmente, decidimos calcular los pesos asignados a los activos mediante simulaciones de Montecarlo. Aunque la opción de minimización de la varianza según las correlaciones daba resultados similares en los backtests, estábamos más familiarizados con el funcionamiento y justificación teórica del método de Montecarlo, cubierto por el programa del máster [1]. Con este método, estamos aprovechando los grados de libertad que nos aporta la elección de los pesos de los activos en cartera para maximizar el ratio sharpe. Además, conseguimos mitigar, en cierto modo, la influencia que tiene el modelo de ML utilizado para la selección de activos en los retornos de la cartera.

2. Metodología aplicada

2.1. Modelos ML

Explicar lo que es un árbol de decisión, qué parámetros tiene, etc (citar apuntes de Manuel Rueda). A continuación, explicar los principales modelos de ML. Explicar con más detalle cómo funciona el gradient boosting y por qué es el modelo elegido para nuestra estrategia: hay razones cualitativas pero también porque probamos varios en un backtest y era el que mejor tiraba. También explicar cómo se entrena el modelo.

Para llevar a cabo nuestra estrategia de selección de activos (*stockpicking*) sobre el índice EuroStoxx50, es fundamental enmarcar metodológicamente el problema dentro de las técnicas de aprendizaje automático.

2.1.1. Tipo de aprendizaje y enfoque del problema

Atendiendo a la naturaleza de los datos y al objetivo de nuestra estrategia de inversión, este desarrollo se encuadra dentro del **Aprendizaje Supervisado**. Utilizamos datos históricos provistos por YahooFinance perfectamente etiquetados con el resultado correcto (rendimientos pasados de las compañías del índice) para entrenar nuestro algoritmo.

Dependiendo de la definición exacta de la variable objetivo en el diseño de la infraestructura, el problema se puede plantear bajo dos enfoques diferenciados:

- **Enfoque de Regresión:** Si la variable respuesta que el modelo intenta predecir es numérica y continua (como la rentabilidad exacta esperada de cada activo o los cambios futuros en el precio). Las métricas clave para monitorizar su error son el Error Cuadrático Medio (MSE), el Error Absoluto Medio (MAE) y el coeficiente de determinación (R^2).
- **Enfoque de Clasificación:** Si la variable respuesta se define como una etiqueta categórica binaria (por ejemplo, clasificar un activo con un “1” si se predice que batirá al índice para proceder a su compra, o con un “0” en caso contrario). Bajo este enfoque, la evaluación del modelo se realiza a través de la matriz de confusión, monitorizando la tasa de aciertos (*Accuracy*), la Precisión, el *Recall*, el indicador *F1-score* y el análisis de la curva ROC (AUC).

2.1.2. Modelo seleccionado: XGBoost (Extreme Gradient Boosting)

Los árboles de decisión individuales son algoritmos propensos a una alta varianza y propicios al sobreajuste (*overfitting*). Por ello, para maximizar el poder de generalización del modelo en mercados financieros, se ha optado por un método de ensamblado (*Ensemble Learning*), concretamente un algoritmo secuencial de **Boosting: XGBoost (Extreme Gradient Boosting)**.

La elección de XGBoost frente a otros métodos (como las técnicas de promedio o *Bagging* como *Random Forest*) se fundamenta en los siguientes pilares teóricos extraídos de la literatura y el material docente:

1. **Reducción del Sesgo (Bias Reduction):** A diferencia de los métodos de *Bagging* —donde los árboles se entrenan en paralelo y cada registro tiene las mismas posibilidades de aparecer en el *dataset*—, XGBoost entrena estimadores débiles de forma secuencial. Cada nuevo árbol se enfoca de manera específica en corregir los errores o las observaciones más difíciles que fueron peor clasificadas o predichas por los modelos anteriores, asignándoles un peso dinámico superior. Esto minimiza el sesgo general del estimador.

2. **Regularización Integrada contra el Overfitting:** Aunque por definición los algoritmos de *Boosting* corren el riesgo de incrementar el sobreajuste si se añaden demasiados estimadores, XGBoost destaca por incorporar penalizaciones formales en su función de pérdida (regularización L1 y L2). Esto controla la complejidad de los árboles internos y evita que el algoritmo aprenda el “ruido” de los datos financieros, garantizando un rendimiento más robusto durante la fase posterior de *backtesting*.
3. **Eficiencia y Computación en Paralelo:** A nivel de infraestructura de código, XGBoost optimiza drásticamente los recursos computacionales al permitir la paralelización durante la construcción de los árboles de decisión, lo que reduce los tiempos de ejecución notablemente en comparación con el *Gradient Boosting* tradicional.

2.2. Evaluación. Métricas de rendimiento y riesgo

Antes de simular la estrategia en el *backtest*, el conjunto de datos fue dividido formalmente para mitigar el riesgo de *overfitting* e *infra-ajuste* (*underfitting*), asegurando que el modelo retenga la capacidad de generalizar con datos no vistos en el entrenamiento. El rendimiento final de XGBoost se monitoriza contrastando las métricas de error y acierto predictivo en el subconjunto de test y validación cruzada, garantizando así una base sólida previa a la optimización final de la cartera.

2.3. Desagregación de resultados

A la hora de calcular los resultados, es necesario realizar una desagregación de los resultados para realizar un análisis más completo del desempeño de la cartera. Esto cobra un especial sentido al tratarse de una estrategia de gestión cuantitativa, en la que las decisiones de inversión no se toman de manera directa. Como sabemos, los rebalanceos semanales de la cartera se realizan según un criterio puramente cuantitativo y complejo. Esta complejidad diluye la transparencia de la toma de decisiones, así que tratamos de analizar los movimientos en detalle para aprender sobre el modelo y sus puntos débiles.

La desagregación más sencilla y natural es el *performance attribution*, es decir, el detalle de la rentabilidad o P&L netos por cada activo en el que hemos estado invertidos. Su cálculo es inmediato: sería la rentabilidad compuesta de un activo en cada periodo, calculada a partir de los precios de ejecución y el número de acciones en cartera en cada fecha de dicho activo. Todos estos datos han sido cuidadosa y metódicamente registrados en un csv con cada operación de compra-venta.

Aunque lo anterior nos da una visión del peso de cada activo elegido en el P&L final de la cartera, no aporta demasiada información sobre el desempeño del modelo de selección de activos. Este modelo trata de predecir los activos que van a realizar los mayores retornos en comparación con el resto de integrantes del índice, lo cual nos conduce a la pregunta: ¿acierta el modelo? Para ello, debemos fijarnos en los resultados semanales. En este punto, nos parece de interés mostrar, para cada semana, el puesto por rentabilidad que ha conseguido cada activo elegido (al inicio de la semana).

Lo comentado anteriormente nos habla de la capacidad de predicción del modelo, pero no sobre el efecto (en rentabilidad) que ha tenido la selección de activos. En este punto, nos vimos en la tesitura de elegir una métrica que refleje adecuadamente este efecto. Para ello, decidimos centrarnos en la métrica $\alpha = R_{\text{cartera}} - R_{\text{bmk}}$, ya que el objetivo de esta estrategia es batir al *benchmark*. Ahora, definimos las siguientes variables, utilizando el subíndice i para denotar el activo:

$$es_i = \omega_{bmk,i}(r_i - R_{bmk}), \quad ep_i = (\omega_{c,i} - \omega_{bmk,i})(r_i - R_{bmk}), \quad (1)$$

donde $\omega_{bmk,i}$, $\omega_{c,i}$ son los pesos del activo en el EuroStoxx50 y la cartera, respectivamente, R_{bmk} es la rentabilidad semanal del índice y r_i es la rentabilidad bruta semanal del activo¹ (es decir, usando los precios de cierre y los dividendos, sin costes de operación).

¹Es importante notar, en este punto, que en todo momento estamos utilizando rentabilidades simples no anualizadas.

La variable es_i se puede entender como un “efecto de selección”, ya que mide simplemente la rentabilidad del activo relativa al índice, ponderado por su peso en el mismo. Así, los activos tendrán un efecto de selección grande si superan sobradamente al *benchmark* y negativo si no lo hacen. Por otro lado, ep_i lo interpretamos como el efecto de los pesos elegidos para la cartera. La razón está en que esta métrica devuelve valores altos cuando un activo que supera al índice está sobreponderado con respecto al benchmark o un activo con rentabilidad inferior está infraponderado (y viceversa). Es decir, es una especie de medida de cómo, elegidos unos activos, se asignan los pesos para maximizar la rentabilidad.

A partir de las definiciones, podemos comprobar fácilmente que

$$\begin{aligned} es + ep &= \sum_i es_i + \sum_i ep_i = \sum_i \omega_{bmk,i}(r_i - R_{bmk}) + \sum_i (\omega_{c,i} - \omega_{bmk,i})(r_i - R_{bmk}) \\ &= \sum_i \omega_{bmk,i} \cdot r_i - R_{bmk} + \sum_i \omega_{c,i} \cdot r_i - R_{bmk} - \sum_i \omega_{bmk,i} \cdot r_i + R_{bmk} \\ &= R_c^{bruta} - R_{bmk}, \end{aligned} \quad (2)$$

donde hemos usado que la suma de los pesos verifica $\sum_i \omega_{bmk,i} = \sum_i \omega_{c,i} = 1$, y además la rentabilidad bruta de la cartera es la suma de los productos de las rentabilidades (sin costes) y los pesos en la cartera $R_c^{bruta} = \sum_i \omega_{c,i} \cdot r_i$. En consecuencia, si denotamos los costes operativos como ec , tenemos:

$$\begin{aligned} \alpha &= R_{cartera} - R_{bmk} = ec + R_c^{bruta} - R_{bmk} = ec + es + ep \\ &\Rightarrow R_{cartera} = R_{bmk} + ec + es + ep. \end{aligned} \quad (3)$$

Recapitulando, hemos definido tres efectos (selección, peso y costes) de manera que somos capaces de desagregar la rentabilidad semanal de la cartera en la suma de estos efectos y la rentabilidad del benchmark, a la cual nos referiremos como “efecto de mercado”.

Hemos podido comprobar que estas métricas no solo permiten un análisis de la rentabilidad con detalle a nivel activo, sino que cada una de ellas tiene un sentido financiero perfectamente definido. Además, nuestra estrategia se puede dividir a muy grandes rasgos en dos partes: el modelo de machine learning realiza la selección de activos, mientras que los pesos se eligen en un proceso separado. Esta lógica nos permite separar las aportaciones de cada una de estas fases.

El razonamiento anterior ha sido exclusivamente centrado en el rendimiento en una sola semana, ya que los cálculos son más sencillos debido a que los pesos no varían en ese periodo. Sin embargo, también será necesario aplicar este análisis a todo el periodo de operación de la cartera. Esto es algo más complejo porque hay que componer los resultados de cada semana en un proceso iterativo.

Denotamos POS_k, BMK_k a los valores liquidativos al final de la k -ésima semana y $R_{c,k-1}, R_{bmk,k-1}$ sus rentabilidades simples durante esa semana² de nuestra cartera y una cartera que replica el índice, respectivamente. En este caso, estamos asumiendo que $POS_0 = BMK_0 = 10.000.000\text{€}$. De esta manera, para cada semana se verifica

$$POS_k = POS_{k-1}(1 + R_{c,k}), \quad BMK_k = BMK_{k-1}(1 + R_{bmk,k}). \quad (4)$$

Por otro lado, denotamos el alpha acumulado (en euros) durante la k -ésima semana como

$$A_k = POS_k - BMK_k = POS_{k-1}(1 + R_{c,k}) - BMK_{k-1}(1 + R_{bmk,k}). \quad (5)$$

Sumando y restando $BMK_{k-1}(1 + R_{c,k})$, llegamos a

$$A_k = (POS_{k-1} - BMK_{k-1})(1 + R_{c,k}) + BMK_{k-1}(R_{c,k} - R_{bmk,k}) = A_{k-1}(1 + R_{c,k}) + BMK_{k-1}\alpha_k. \quad (6)$$

La elección de rentabilidades simples es importante para poder sumar las de los activos dentro de un mismo periodo, lo cual cumplirá propiedades deseables.

²Estamos considerando una semana como la unidad mínima de tiempo porque la estrategia tiene rebalances semanales, pero el razonamiento es válido para cualquier intervalo de tiempo.

Finalmente, como ya hemos visto que el α de un periodo se puede expresar como la suma de los efectos $\alpha_k = ec_k + es_k + ep_k$ (3), hemos llegado a una expresión recursiva:

$$A_k = A_{k-1}(1 + R_{c,k}) + BMK_{k-1}(ec_k + es_k + ep_k) = EC_k^{ac} + ES_k^{ac} + EP_k^{ac}, \quad (7)$$

donde $EC_k^{ac}, ES_k^{ac}, EP_k^{ac}$ son los efectos acumulados de costes, selección y pesos, respectivamente. Es importante notar que estos efectos acumulados no son rentabilidades como ec_k, ep_k, es_k , sino valores absolutos en euros. Se definen iterativamente como sigue:

$$\begin{aligned} EC_1^{ac} &= BMK_0 \cdot ec_1, & EC_k^{ac} &= EC_{k-1}^{ac}(1 + R_{c,k}) + BMK_{k-1} \cdot ec_k & \forall k \geq 2, \\ ES_1^{ac} &= BMK_0 \cdot es_1, & ES_k^{ac} &= ES_{k-1}^{ac}(1 + R_{c,k}) + BMK_{k-1} \cdot es_k & \forall k \geq 2, \\ EP_1^{ac} &= BMK_0 \cdot ep_1, & EP_k^{ac} &= EP_{k-1}^{ac}(1 + R_{c,k}) + BMK_{k-1} \cdot ep_k & \forall k \geq 2. \end{aligned} \quad (8)$$

En conclusión, hemos definido una desagregación en términos de los efectos de mercado, costes, selección de activos y elección de pesos. Este sistema funciona a nivel semanal y por activo, pero también se puede componer para obtener datos agregados del periodo completo.

3. Sistema e infraestructura del modelo

Ya hemos descrito en detalle la idea de la estrategia y su funcionamiento, pero sin entrar en la implementación técnica de la misma. En este respecto, hemos decidido programar la estrategia en Python debido a que se trata de un lenguaje de programación versátil y potente, pero también con el que estamos familiarizados todos los integrantes del grupo. En concreto, decidimos utilizar programación orientada a objetos definiendo varias clases modulares complementarias. Creemos que esta manera de trabajar es la idónea en este tipo de proyectos, ya que permite dividir el proyecto en módulos “independientes” y los miembros del grupo podemos trabajar paralelamente en clases diferentes del código para después compartir los avances mediante un repositorio de GitHub.

3.1. Estructura del bloque de código principal

Las clases principales del sistema se organizan en varios módulos funcionales, tal y como se resume en el esquema de la fig. 1. En términos generales, el código se puede dividir en cuatro bloques: un bloque de datos y universo, encargado de definir los activos disponibles y descargar la información de mercado; un bloque de construcción de variables, que transforma los datos brutos en variables explicativas y variable objetivo; un bloque de modelo y decisión de inversión, donde se entrena el modelo de machine learning y se convierten sus predicciones en una cartera objetivo; y, finalmente, un bloque de ejecución y evaluación, que permite tanto simular la estrategia históricamente como generar señales reales de inversión. A continuación, se describe brevemente la función de cada fichero principal del código:

- **UniversoActivos.py.** Este fichero contiene las clases encargadas de definir el universo de inversión. Definimos una clase padre que establece la interfaz común. A partir de ella, se implementan dos versiones: **UniversoActivosEstatico**, que mantiene constante la lista de activos durante todo el periodo, y **UniversoActivosDinamico**, que reconstruye el universo disponible en cada fecha a partir de los cambios históricos en la composición del índice. La primera se utiliza para implementar el modelo (no se necesitan cambios históricos), mientras que la segunda es imprescindible para realizar backtests imparcialmente (*survivorship bias*).
- **ProveedorDatos.py.** Este módulo se encarga de la obtención y preparación inicial de los datos de mercado. Hay una clase padre **ProveedorDatosBase** con una clase hija **YFinanceProvider**, basada en la librería **yfinance**. El proveedor descarga de *Yahoo Finance* precios de cierre, volumen negociado y dividendos. Además, construye dos bases de datos: una diaria, utilizada para valorar la cartera y calcular determinados indicadores; y una semanal, utilizada por la estrategia para determinar las órdenes de rebalanceo.
- **VariablesTransformation.py.** Este fichero contiene la clase **FeatureEngineer**, responsable de transformar los datos de mercado en una base apta para el entrenamiento del modelo. En este módulo se calculan las variables explicativas con las que se entrena el modelo de ML. Son medidas de momentum, volatilidad, liquidez, drawdown, beta frente al índice, RSI, MACD, medias móviles y bandas de Bollinger. También se construye la variable objetivo del problema de clasificación, que identifica si un activo pertenece al grupo de mejores activos durante la semana siguiente.
- **Modelos.py.** Este módulo contiene la lógica de entrenamiento de los modelos de machine learning. Se define la clase base **ModeloBase**, que recoge la estructura de entrenamiento y predicción de probabilidades que deben tener todos los modelos. Sobre ella, se implementan modelos concretos como Random Forest y Gradient Boosting. Aunque finalmente decidimos utilizar un modelo de Gradient Boosting (XGBoost), se mantienen las clases correspondientes a otros modelos probados. Por otro lado, hemos creado la clase **WalkForwardCV** para implementar una validación cruzada temporal (*k-fold walk-forward cross validation*). Esto fue necesario en modelos de series temporales porque respeta el orden cronológico de los datos y evita entrenar con información futura. Además, al contrario que otros métodos de cross validation más sencillos, no está implementado en ninguna librería.

- **Estrategia.py.** Este fichero transforma las predicciones del modelo en decisiones de inversión. La clase base **EstrategiaBase** define la interfaz común de las estrategias, mientras que las clases derivadas implementan distintas formas de seleccionar activos y asignar pesos. La estrategia equiponderada selecciona los activos con mayor score y reparte el peso por igual. La estrategia basada en Montecarlo, que es la más representativa de la versión final, utiliza el modelo para preseleccionar activos y después optimiza los pesos mediante simulaciones, buscando maximizar el ratio Sharpe esperado. También se implementan variantes con señal macro y con optimización basada en alpha, riesgo y costes de transacción.
- **Backtest.py.** Este módulo contiene el motor de simulación histórica o *backtest*. La clase principal (**BacktestEngine**) simula el funcionamiento hipotético de la estrategia en un intervalo pasado: descarga los datos necesarios, construye las variables disponibles en cada fecha, reentrena el modelo cuando corresponde, genera la cartera objetivo, ejecuta los rebalances semanales, aplica costes de transacción, incorpora dividendos y valora diariamente la cartera. También se añadió una clase auxiliar **BacktestRandom** para realizar backtests aplicando estrategias aleatorias. El objetivo era comparar el desempeño de nuestra estrategia contra una estrategia que sigue la misma lógica pero tomando decisiones aleatorias.
- **MotorInversion.py.** Aquí se implementa el motor operativo de la estrategia, es decir, la parte pensada para su uso fuera del backtest. La clase **MotorInversion** carga la cartera actual, descarga los datos más recientes, actualiza el valor actual de la cartera, reentrena el modelo si corresponde y genera las señales de compra, venta o mantenimiento. Además, guarda el estado actualizado de la cartera, el historial de operaciones y el modelo entrenado. Esto no solo es necesario para volver a aplicar la estrategia la semana siguiente (guarda la última posición), también facilita el proceso de enviar el excel de operaciones: únicamente hay que copiar las filas de la fecha correspondiente del historial.
- **auxiliary_functions.py.** En este fichero se recogen todas las funciones auxiliares utilizadas en las clases mencionadas anteriormente, manteniendo el código algo más eficiente y limpio. Incluye funciones para calcular indicadores técnicos, como RSI, MACD, bandas de Bollinger o beta; calcular costes de transacción; valoración de cartera *mark-to-market*; y construir tablas de métricas de performance. Aunque no constituye un bloque independiente dentro del esquema principal, sirve como soporte común para el resto de módulos.

Esta organización permite que cada fichero tenga una responsabilidad y efecto claramente delimitados. Esto facilita el mantenimiento del proyecto y permite modificar una parte concreta del sistema sin alterar el resto. Por ejemplo, se podría sustituir el proveedor de datos, probar un nuevo modelo de clasificación, cambiar la lógica de selección de activos o modificar el método de asignación de pesos manteniendo intacta la estructura general del código.

3.2. Estructura de código de análisis de resultados

Al aplicar la estrategia para generar las órdenes de rebalanceo semanales, se generan automáticamente los registros de las operaciones realizadas y se actualizan las acciones en cartera. Esto es necesario para realizar el siguiente rebalanceo y facilita el envío de órdenes, pero también lo podemos usar para reconstruir y analizar los resultados de la cartera de cara a la presentación de resultados.

Hemos decidido realizar este análisis de resultados de manera totalmente independiente al bloque principal. De esta manera, reducimos la probabilidad de cometer errores debidos a la creciente complejidad del código. Además, utilizamos como fuente de información un documento .csv que recoge las operaciones que se han enviado por correo y, por tanto, han sido ejecutadas. Por tanto, realizaremos los análisis usando como fuentes de información únicamente los datos históricos de precios de cierre diarios y dividendos (obtenidos en *Yahoo Finance*) y el histórico de operaciones recogido en el documento mencionado.

Todo el código relacionado con el análisis de resultados y riesgo de mercado se encuentra en la carpeta “Monitoring”. Recogemos todas las funciones necesarias en un fichero (`auxfun.py`): descarga de datos, cálculo de rentabilidades y métricas, presentación de resultados en gráficas y tablas, etc. Las funciones de descarga de datos y cálculo de métricas de desempeño son sencillas, tanto su código como conceptualmente. Sin embargo, la parte del código encargada del formato de las tablas y gráficos es mucho más compleja. Consideramos que merece la pena invertir tiempo y esfuerzo en que sean visualmente atractivas tanto en las diapositivas como en este documento. Sin embargo, esta parte del código se ha obtenido con la ayuda de varios LLM. Los resultados están recogidos en el notebook `resultados.ipynb`.

Además de las métricas usuales de rendimiento de la cartera, nos pareció relevante estimar el riesgo de la cartera a través de métricas como el VaR y TailVaR. La metodología aplicada se explica en detalle en la sección 2.2. Las funciones utilizadas para estimar estas variables se encuentra en `var_funciones.py`, y se utiliza en el notebook `var_tailvar.ipynb`.

CONTINUAR...

De manera paralela, nos pareció interesante implementar el análisis de la cartera en un dashboard en tiempo real como un proyecto en paralelo que aporta valor a la simulación

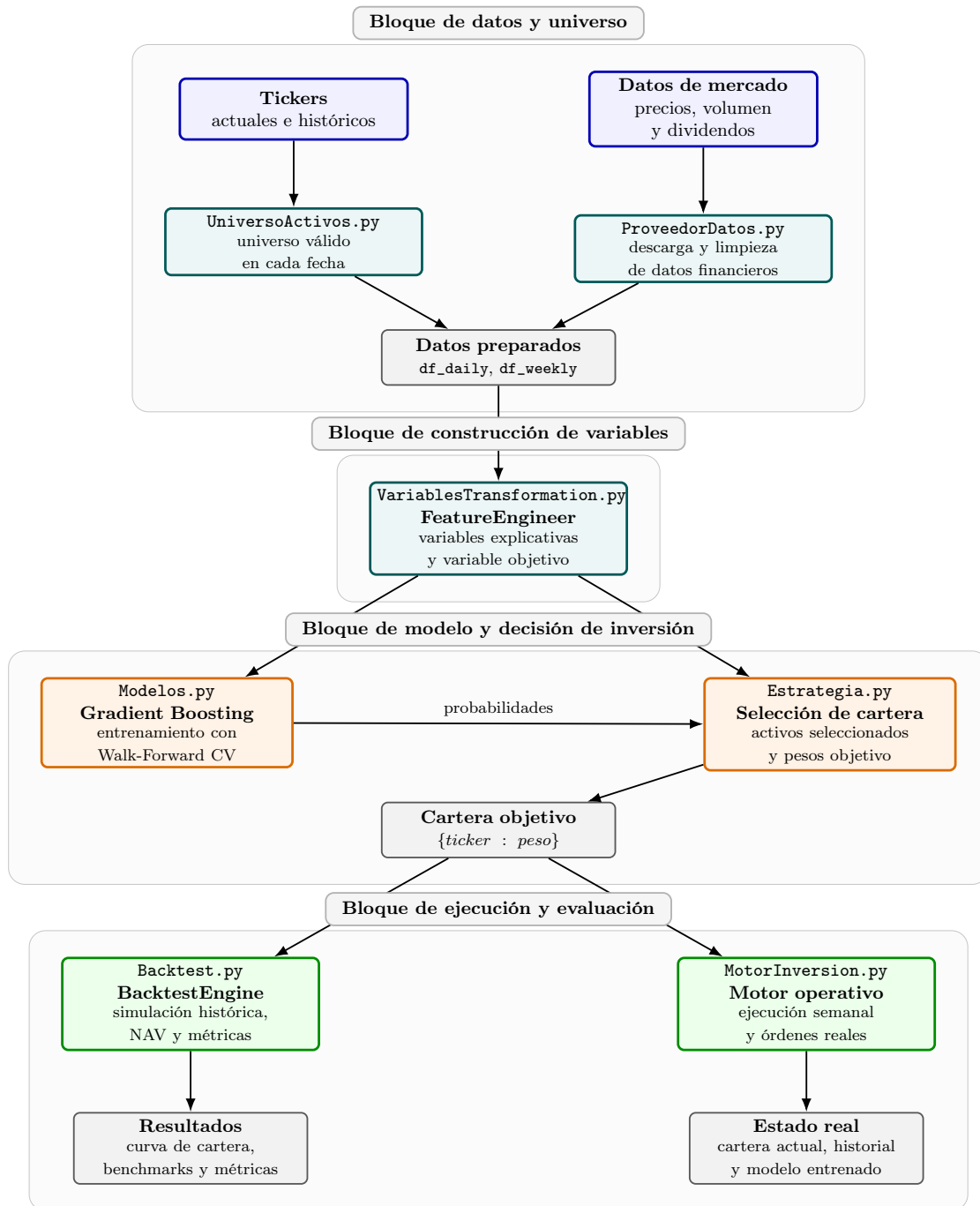


Figura 1: Arquitectura del sistema de clases que implementa la estrategia de gestión cuantitativa.

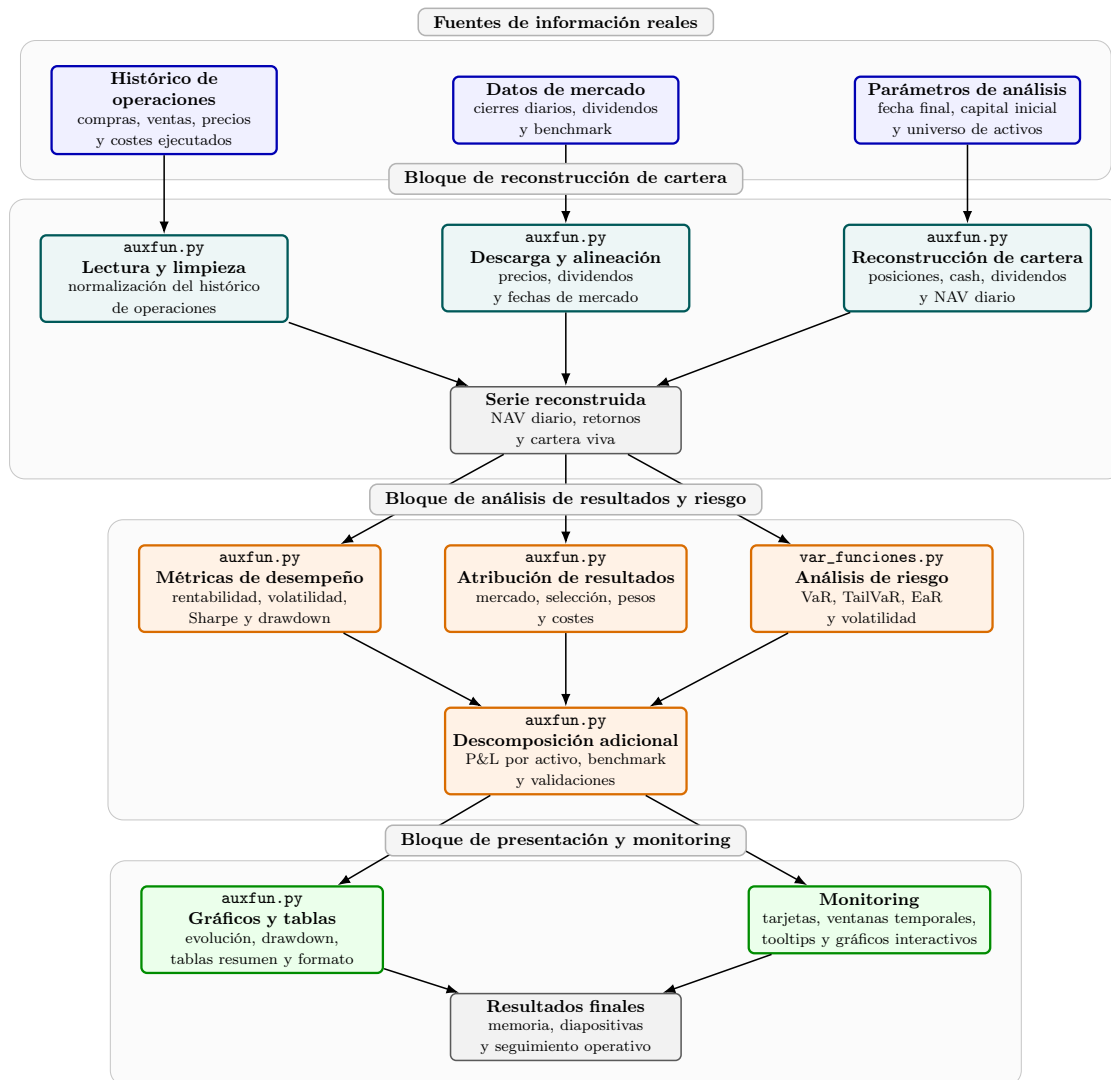


Figura 2: Arquitectura del sistema independiente de monitorización, análisis de resultados y control de riesgo.

4. Backtests e implementación de la estrategia

Después de definir la estrategia a seguir e implementar la infraestructura de código necesaria para ejecutar el modelo, es imprescindible testear el modelo ejecutando una serie de backtests. Esta es una práctica común para simular el desempeño de una estrategia o pautas de inversión en diferentes periodos históricos. Además, estamos considerando varios modelos de Machine Learning y estrategias de selección de pesos que, mientras que no afectan al funcionamiento de la estrategia conceptualmente, pueden resultar ser más o menos adecuados para gestionar esta cartera. Es decir, también usaremos el backtest para elegir estas características del modelo.

Por otro lado, existen una serie de hiperparámetros conceptualmente diferentes a los hiperparámetros internos a los modelos de Machine Learning: la longitud de la ventana de entrenamiento del modelo, el criterio de definición del Target, el número de simulaciones de Montecarlo para la selección de pesos, el número de activos en cartera, el umbral de activos para la rotación, etc. Algunos de estos parámetros se fijaron por criterios objetivos de gestión, y otros fueron fijados según su desempeño en el backtest. Se pueden consultar los parámetros elegidos en la ...

El resto de backtests realizados para diferentes ventanas temporales están en el anexo

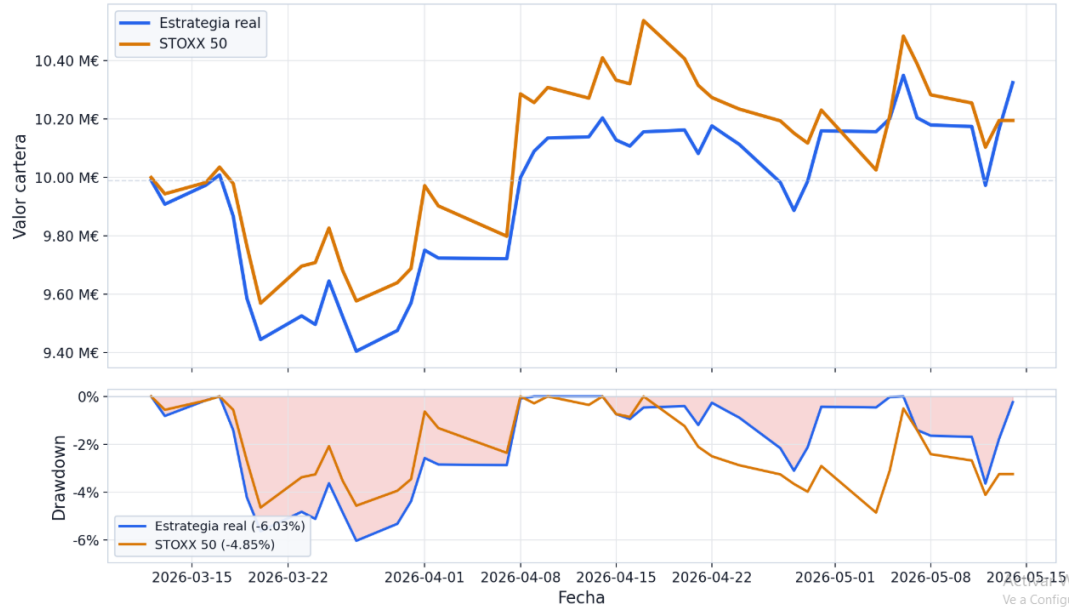


Figura 3: Valor diario (arriba) y *max drowdown* (abajo) de la cartera y el índice EuroStoxx50 partiendo de un capital inicial de 10 millones de euros, entre el 12/03/26 y 14/05/26. Fuente: elaboración propia, YahooFinance.

5. Resultados y análisis de la estrategia

La simulación en directo de la estrategia comenzó el día 12 de marzo de 2026, tras una presentación de la estrategia que implementaríamos. En la primera versión de la estrategia, utilizamos un modelo de Random Forest para seleccionar las 15 acciones en las que invertir, tomando pesos equiponderados. Durante estas primeras semanas, la cartera obtuvo una rentabilidad del -2,76 %, inferior al -2,01 % que hizo el EuroStoxx50 en ese mismo periodo.

Tras una serie de pruebas y backtests con diferentes modelos y estrategias de elección de pesos, decidimos ajustar el modelo hacia su versión final, descrita en detalle en este documento. Durante el resto de las semanas, mantuvimos el nuevo modelo sin cambios destacables, logrando una rentabilidad del 6,39 % frente a la rentabilidad del 4,04 % del índice durante las últimas semanas.

Si tenemos en cuenta la simulación al completo, la estrategia logró una rentabilidad total del 3,25 %, frente al 1,95 % del EuroStoxx50 (fig. 3). Como se puede observar en el gráfico, la estrategia fue capaz de recoger de manera aproximada el crecimiento del mercado, medido a través del *benchmark*. A pesar del alpha negativo de las primeras semanas, la cartera va replicando las rentabilidades del índice con la selección estratégica de 15 activos de los 50 que lo conforman.

A pesar de haber realizado un *max drowdown* peor que el índice (-6,03 % vs. -4,85 %) tabla 1, la volatilidad realizada de la estrategia (18,9 %) es inferior a la del índice, a pesar de estar menos diversificada. Como consecuencia de la mayor rentabilidad y menor volatilidad, se observa un ratio de *sharpe* de 1,03 frente al ratio de 0,54 del benchmark. Estos resultados son congruentes con la decisión de ponderar los activos de la cartera que maximicen el binomio rentabilidad-riesgo, como se explica en la Sección .

Aunque hemos comentado los resultados totales de la cartera, creemos que es importante analizar más en detalle su desempeño. En concreto, sería conveniente desagrupar los resultados en diferentes aportaciones de cada pieza que conforma nuestra estrategia.

La estrategia se fundamenta fuertemente en un algoritmo que trata de predecir el desempeño de una serie de activos, pero durante el entrenamiento del modelo podemos observar que sus métricas de acierto son solo ligeramente superiores a la aleatoriedad. Esto implica que la estrategia no tiene por qué mostrar resultados a corto plazo, su atractivo reside en que, de media, va a elegir los activos mejor que como lo hace el índice (que es prácticamente una equiponderación de los 50 activos). De esta manera, los rendimientos ligeramente superiores al índice se irán componiendo y haciendo “bola de nieve”, y es a largo plazo cuando se hace aparente la verdadera superioridad de la estrategia. Hemos podido simular y comprobar el desempeño de la estrategia en backtests durante diferentes ventanas que hemos comentado en la Sección section 4 (referenciar una de las gráficas de backtest).

Tabla 1: Tabla de métricas de la estrategia y del *benchmark* durante la simulación (del 12/03/26 al 14/05/26). Fuentes: elaboración propia, YahooFinance.

Métrica	Estrategia	STOXX 50
Rentabilidad	3.25 %	1.95 %
Volatilidad	18.89 %	22.55 %
Sharpe	1.03	0.54
Max DD	-6.03 %	-4.85 %

6. Conclusiones

Añadir algunos párrafos de aspectos a mejorar o que podrían mejorarse en caso de tener herramientas más potentes o más tiempo: grid de entrenamiento del modelo, variables utilizadas, número de simulaciones de montecarlo...

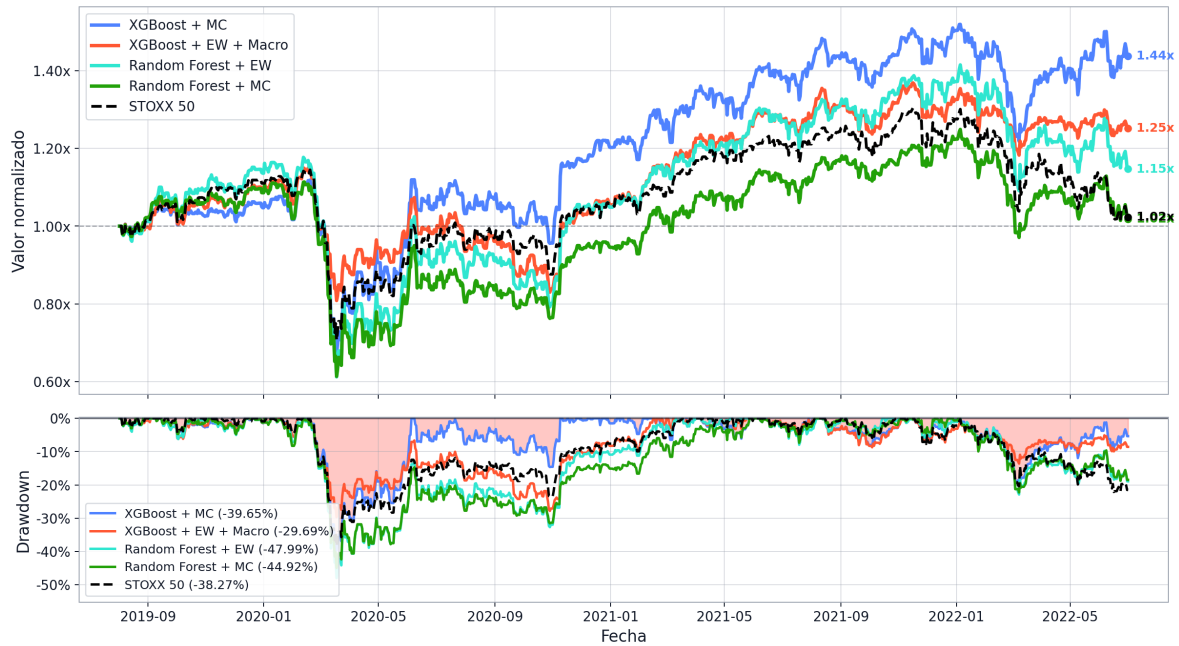


Figura 4: Valor diario (arriba) y *max drawdown* (abajo) de la cartera y el índice EuroStoxx50 partiendo de un capital inicial de 10 millones de euros. Resultado de un backtest entre el 01/08/19 y 01/07/22. Fuente: elaboración propia, YahooFinance.

Estrategia	Rentabilidad	Volatilidad	Sharpe	Max DD
XGBoost + MC	43.80 %	22.20 %	0.50	-39.65 %
XGBoost + EW + Macro	25.14 %	17.32 %	0.39	-29.69 %
Random Forest + EW	14.70 %	25.71 %	0.26	-47.99 %
Random Forest + MC	1.81 %	23.75 %	0.14	-44.92 %
STOXX 50	2.33 %	20.15 %	0.13	-38.27 %

Tabla 2: Métricas de las distintas estrategias en un backtest entre el 01/08/19 y 01/07/22. Fuente: elaboración propia, YahooFinance.

7. Anexo

7.1. Backtest

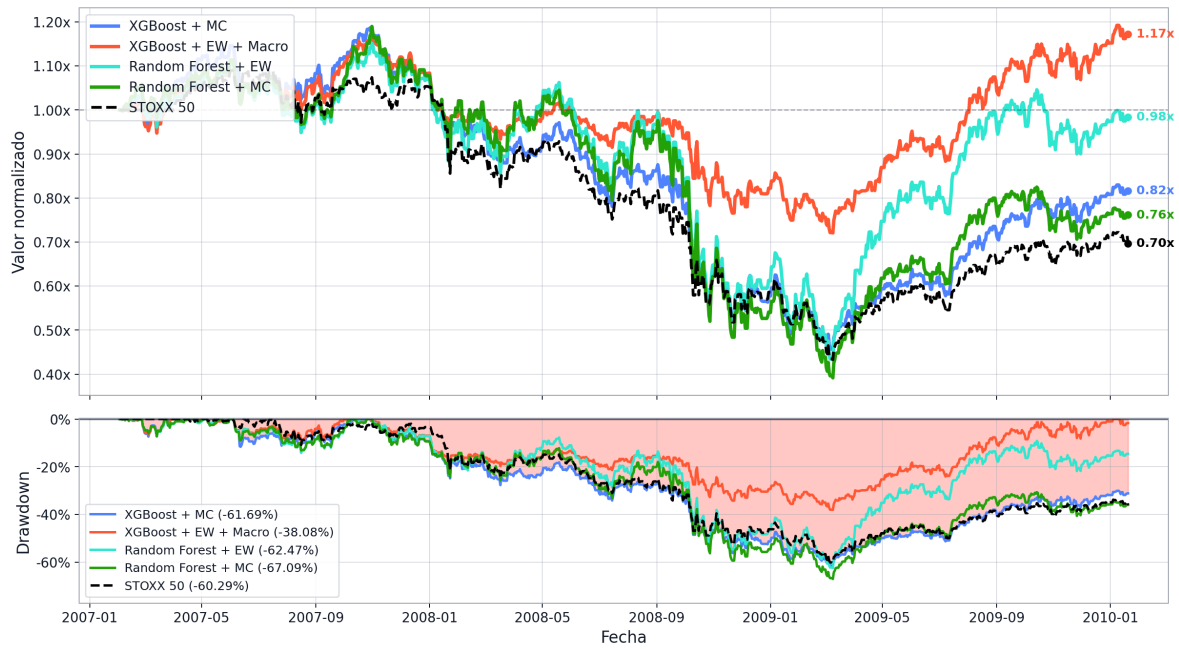


Figura 5: Valor diario (arriba) y *max drawdown* (abajo) de la cartera aplicando varias estrategias y el índice EuroStoxx50 partiendo de un capital inicial de 10 millones de euros. Resultado de un backtest entre el 31/01/2007 y 20/01/2010. Fuente: elaboración propia, YahooFinance.

Estrategia	Rentabilidad	Volatilidad	Sharpe	Max DD
XGBoost + MC	-18.33 %	27.49 %	-0.04	-61.69 %
XGBoost + EW + Macro	17.24 %	17.02 %	0.30	-38.08 %
Random Forest + EW	-1.64 %	31.72 %	0.15	-62.48 %
Random Forest + MC	-23.84 %	31.50 %	-0.04	-67.09 %
STOXX 50	-30.29 %	24.34 %	-0.22	-60.29 %

Tabla 3: Métricas de las distintas estrategias en un backtest en una ventana alrededor de la crisis financiera de 2008: entre el 31/01/2007 y 20/01/2010. Fuente: elaboración propia, YahooFinance.

Referencias

- [1] Jose Javier Calderón. *Métodos Numéricos. Optimización*. 2026.
- [2] Jose María Contreras y Adrián González. *Gestión Cuantitativa*. 2026.